Chess Ranger Report

American University of Armenia

Zaven & Sonia Akian College of Science & Engineering

CS246 - Artificial Intelligence and Decision Support

Group: Natali Arzumanyan, Arno Barzegarnazari,

Mikayel Harutyunyan, Davit Ter-Harutyunyan

Instructor: Vahram Tadevosyan

December 8, 2025

Abstract

The Chess Ranger puzzle is a logic problem that combines the strategies of chess with analytical challenges of logical puzzles. The puzzle starts with a few chess pieces placed on a standard chess board. On the board, only capture moves are allowed, including capturing the king. To solve this puzzle, only a single piece has to be on the board. This project explores the Chess Ranger puzzle as a search problem, aiming to explore how different search algorithms can find solutions and how to optimize the search process. Specifically, it focuses on four main approaches: DFS (depth-first search), A* search (A-star), Hill-Climbing search, and Simulated Annealing search and compare their effectiveness in handling discrete, deterministic environments. This report presents a literature review of the problem and solutions, outlines the methods used to solve the problem, and discusses the results and analyzes them. The end goal is to compare the algorithmic performance and provide insights into optimizations of this specific puzzle.

Keywords: Artificial Intelligence, Chess, puzzle, Chess Ranger, algorithm, heuristic, local search, uninformed search, informed search, heuristic function, A*, Hill-Climbing, Stochastic-Hill-Climbing, Simulated Annealing, DFS

Contents

Abstract **2**
Contents **3**
1 Introduction **4**
1.1 Chess Ranger 4
1.2 Rules of the Game 4
1.3 Motivation 5
1.4 Problem Setting and Description 5
1.4.1 Formal Formulation of the Problem 6
1.5 Our Focus and Goal 7
1.6 Paper Structure 7
2 Literature Review **8**
2.1 Solitaire Chess 8
2.2 Solo Chess 8
2.3 “The Isolated” Heuristic 9
3 Method **10**
3.1 Algorithms Considered 10
3.2 Heuristic Functions 10
3.2.1 The Mobility Heuristic 11
3.2.2 Admissibility and Consistency 11
3.3 Designing a GUI 15
4 Experiments, Results and Analysis **16**
4.1 Experiments 16
4.1.1 Small Configuration Experiment 16
4.1.2 Large Configuration Experiment 16
4.2 Results 17
4.2.1 Success Rate 17
4.2.2 Runtime 19
4.2.3 Number of Expanded Nodes 20
4.2.4 Number of Generated Nodes 21
5 Conclusion **22**
5.1 Difficulties Encountered 22
5.2 Future Research Directions 23
6 References **24**

1 Introduction

Artificial intelligence is often related to games and puzzles, where clear rules and outcomes make them ideal testing material for search algorithms. Classic board games like chess and checkers have been used to evaluate algorithm efficiency and decision making strategies. Logic puzzles, on the other hand, provide small but non-trivial environments where heuristic evaluation and optimization can be explored without wasting much time on computations.

1.1 Chess Ranger

The Chess Ranger is a single-player, logical puzzle which is about making the correct capture moves in a given chess piece configuration to solve the puzzle. The puzzle looks simple at first, but despite its straightforward rules, finding the correct sequence of captures that leads to a single remaining piece can be quite challenging. A particular problem instance can be seen in Figure 1, where there are 4 pieces on the standard chess board.

1.2 Rules of the Game

The rules of the game are simple and as follows:

- Each piece moves as a standard chess piece (meaning bishops can only move diagonally, rooks horizontally or vertically, and etc.)
- The player can only perform capturing moves
- The game allows the king to be captured
- End with a single piece on the board

We took the rules of the game from a www.puzzle-chess.com/chess-ranger-4/, where the game can also be played [2].

1.3 Motivation

We chose this project because of a few reasons. The Chess Ranger puzzle combines elements of chess movement, logic puzzles, and search which makes it a perfect environment for exploring optimization strategies. Additionally, no known prior analysis has been conducted on the puzzle creating an opportunity to come up with original findings. Moreover, the “Isolated Heuristic” in Section 2.3 can be improved which can potentially improve search as well. These reasons helped us pick the puzzle for our study.

1.4 Problem Setting and Description

By approaching this puzzle as a search problem, we define every arrangement of the board as a possible state, and each capture represents a state transition. The nature of this puzzle is:

- discrete - finite states and actions
- deterministic - we know the outcome of each action
- fully observable - information about each state is fully available
- known - the rules of the game are fully known in advance
- static - the environment does not change unless a move is made
- sequential - each capture changes the board and affects future possibilities, so long-term consequences must be considered
- finite - number of states may be extremely large, but not infinite

Much like famous problem environments in artificial intelligence such as the 8-puzzle or the n-queens problem.

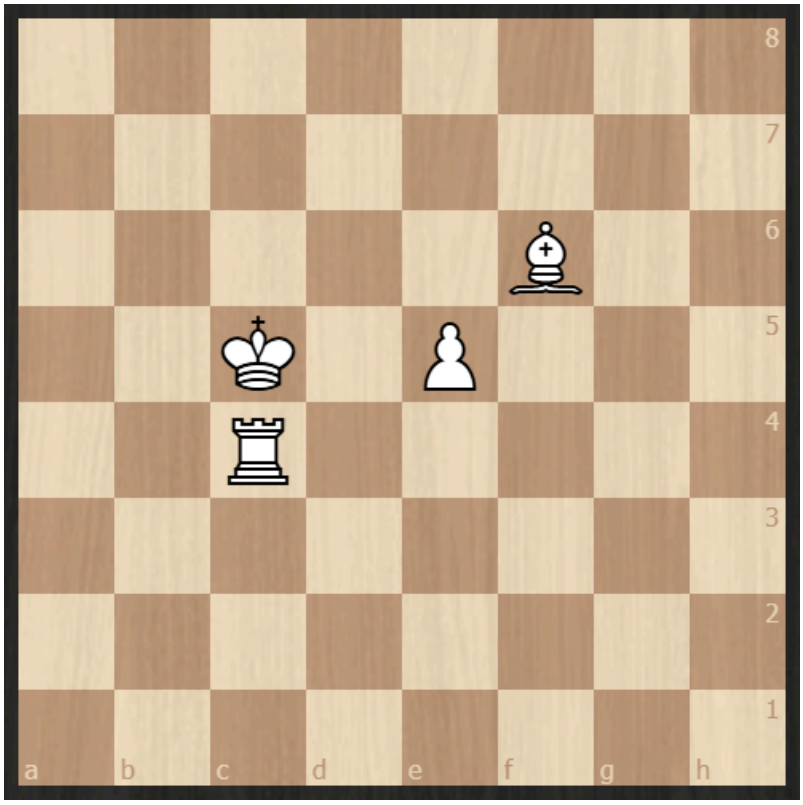

Figure 1: Chess Ranger state example with 4 pieces

1.4.1 Formal Formulation of the Problem

To make our approach complete, we have to formulate the problem formally for search.

State space: s = {p1, p2, …, pk} where pi is every piece on the board. Each piece is identified by their type (king, queen, pawn, and etc.), which row they are on and which column they are on.

The initial state can be any random state from the state space.

The goal states are the states where a single piece is on the board. The goal test is defined as "does the board have 1 piece", if yes, then goal.

The actions are any legal capture move available from that state. The transition is a state after completing the capture. Since there is no difference between actions, a uniform action cost of 1 seems reasonable. The grand cost would be k - 1 which is actually the number of moves required to solve the puzzle with k pieces.

1.5 Our Focus and Goal

Our goal is to test algorithms that can efficiently find the needed sequence of moves to solve the puzzle. To handle all this, we decided to compare different search algorithms, that we explored during this course, such as DFS, A*, Hill-Climbing, and Simulated Annealing. We chose these algorithms since they look at the problem from different angles, hence giving us multiple perspectives and broadness. While, DFS will try every move from each state until no moves are left or a goal is reached, A* gives priority to finding an optimal path by combining the actual cost from the initial state to the current state and the estimated cost from the current state to the goal state. Hill-Climbing focuses more on speed and simplicity, always moving toward what seems like a better state at a certain moment. This means that it can get stuck when there is more than one “better” state. For that, we look at Stochastic Hill-Climbing and Simulated Annealing. By examining these, we hope to understand how different priorities can affect the performance, and whether it is possible to find a middle ground between those two.

1.6 Paper Structure

This is how the report looks like:

- **Section 2** reviews related literature on AI approaches to Chess Ranger solving and other related aspects.
- **Section 3** describes the methods used and the algorithms implemented.
- **Section 4** presents experiments, evaluation metrics, results and analysis.
- **Section 5** concludes findings, difficulties encountered, and future research directions.

2 Literature Review

In this section, we analyze literature regarding this puzzle, whether it is exactly about this puzzle or about something similar. The survey in this section not only includes academic papers or journal articles, also includes similar puzzles, code solutions of chess related puzzles, useful algorithm implementations, and even related interface implementations. Note that we have two sections that talk about problems similar to Chess Ranger (Solo Chess and Solitaire Chess) with slight differences in their rules.

2.1 Solitaire Chess

The Chess Ranger is an example of a puzzle that seems simple but quickly scales up to be computationally challenging with increased board size and number of pieces. One classical version of this puzzle, known as “Solitaire Chess”, was analyzed in a more formal way by Maßberg [3], where he focuses on the case where all pieces are of one color, every move must be a capture, and the goal is to reduce the number of pieces on the board to one. The only difference between Solitaire Chess and Chess Ranger is that Solitaire Chess is played on a 4x4 board while the latter is played on a standard board. Maßberg demonstrated that determining whether a given configuration is solvable is NP-complete. He proves this by reducing the problem to 3-SAT to show that the general case is computationally hard. Therefore, Solitaire Chess is a very good test case for studying heuristic and algorithmic approaches, such as DFS and A* when devising automated solvers for single-player capture puzzles.

2.2 Solo Chess

Solo Chess has the same rules as our Chess Ranger, with the addition that here king cannot be captured. In the paper "Chess Is Hard Even for a Single Player,” Aravind, Misra, and Mittal [1]

discuss computational difficulties of Solo Chess. The authors generalize the game to unbounded boards and fix the limit for each piece to a number of captures. They also prove that the problem having a solvable configuration is NP-complete, even for simpler problems when there are only rooks or queens and at most two captures [1]. They also show that variations like rooks on a one-dimensional board or pawns having only two captures can be solved in linear time, so one can drastically change computational complexities by applying small changes to the rules.

The authors further introduce an associated abstraction called the Graph Capture Game, where pieces correspond to graph vertices and captures are made along edges. They also provide an efficient characterization of solvable pawn configurations in the form of skewed binary trees, showing the dependence of solvability of the game on the type of piece and constraints imposed [1].

2.3 “The Isolated” Heuristic

In our project, we will extend an already quite successful solver that we found on Github, based on DFS and A* search for the solution of configurations of Chess Ranger. In this version, the DFS provides a systematic backtracking exploration of all the possible sequences of captures for completeness, while A* takes a heuristic approach to make the search more efficient. The heuristic h(n) is the number of isolated pieces on the board, representing an estimate of how far the current board state is from the goal of the single remaining piece [4]. What makes pieces harder to eliminate is that they are "isolated," and their capture must come through some particular line or sequence of moves; hence, a board configuration with more isolated pieces is "farther" from its solution.

It is an admissible heuristic because it never overestimates the number of minimum moves, as at least one move per isolated piece is required for its removal. By preferring to go down the branch

where there are fewer isolated pieces, A* greatly reduces the search space and subsequently arrives at solutions more quickly than pure DFS. This makes it a good candidate for a heuristic function. We extended this further by implementing not only DFS and A*, but also Hill-Climbing, Stochastic Hill-Climing, and Simulated Annealing. Hill-Climbing will select moves iteratively, which for a given evaluation function will improve the board configuration, Stochastic Hill-Climbing will try to break unstuck positions where Hill-Climbing can get stuck. Those occur when the evaluation function for two different states is the same. Simulated Annealing probabilistically accepts non-optimal moves to escape local minima. For the evaluation function of the local search method we have used the Mobility heuristic function, which is discussed in Section 3.2.1. Putting all these together, we can explore exhaustive and heuristic-guided strategies, enabling us to construct an adaptable solver that can handle many randomly generated board configurations ranging from easy to very challenging.

3 Method

3.1 Algorithms Considered

Search problem formulation, state space, popular similar problems (8 queens problem) have guided us to select search algorithms as the foundation of this project. As discussed before, our focus will be DFS, A*, Hill-Climbing (and Stochastic), Simulated Annealing and their algorithmic comparison. We have implemented these algorithms using Python.

3.2 Heuristic Functions

In section 2.3 we talked about “The Isolated” heuristic. The heuristic is good, but we believe there is an improvement.

3.2.1 The Mobility Heuristic

Let:

- $k(s)$ = number of pieces in each state.
- I = number of isolated pieces ($d=0$).
- W = number of weakly connected pieces ($d=1$).
- S = number of "strongly" connected pieces ($d \geq 2$).

d is the degree of each piece, which we define as the number of pieces attacking that piece + the number of pieces that piece is attacking.

Then $k(s) = I + W + S$ and the heuristic is defined as $h_mobility(s) = I + \frac{1}{2}(W)$.

3.2.2 Admissibility and Consistency

Lower Bound on True Cost

From any stat with k pieces, every legal move removes exactly one piece (a capture). To go from k pieces to 1, k - 1 moves or captures have to be made. So the optimal solution cost satisfies $C^*(s) \geq k(s) - 1$. If the state is actually unsolvable, no sequence of captures reaches 1 piece, meaning $C^*(s) = \infty$ and any heuristic value is automatically $\leq C^*(s)$. So for proving admissibility, showing that $h_mobility(s) \leq k(s) - 1$ for every non-goal state where a move is possible is enough.

Relating h_mobility to k

We want to show:

$I + \frac{1}{2}(W) \leq k - 1$whenever a capture is possible.

Substitute $k = I + W + S$: $I + \frac{1}{2}(W) \leq I + W + S - 1$

Cancel I from both sides: $\frac{1}{2}(W) \leq W + S - 1$

Rearrange: $0 \leq \frac{1}{2}(W) + S - 1 \Leftrightarrow \frac{1}{2}(W) + S \geq 1$.

So the core combinatorial fact we must show is: In any state where at least one capture is possible, $\frac{1}{2}(W)+S \geq 1$. If that's true, then $I + \frac{1}{2}(W) \leq k - 1$, so $h_mobility(s) \leq k(s) - 1 \leq C^*(s)$, which makes the heuristic admissible.

Proving $\frac{1}{2}(W) + S \geq 1$ when a Capture Exists

Consider the capture-graph GG where:

- vertices = pieces,
- directed edge $u \rightarrow v$ means "u can currently capture v".

If at least one capture is legal in state s, then there is at least one directed edge $u \rightarrow v$ in G.

Now look at the degrees of u and v:

- For u: out-degree (piece attacking pieces) $d(u)+ \geq 1 \Rightarrow d(u) = d(u)+ + d(u)- \geq 1$.
- For v: in-degree (piece attacked by pieces) $d(v)- \geq 1 \Rightarrow d(v) = d(v)+ + d(v)- \geq 1$.

So both u and v have a total degree ≥ 1.

Hence they are not isolated. That means each of them belongs to either:

- the weak set (degree =1), or
- the strong set (degree ≥ 2).

Therefore, the number of non-isolated vertices is at least 2: $W+S \geq 2$.

From this, two cases:

1. If $S \geq 1$, then $\frac{1}{2}(W) + S \geq S \geq 1$
2. If $S = 0$, then all non-isolated vertices are weak, so $W \geq 2$,

   and $\frac{1}{2}(W) + S = \frac{1}{2}(W) \geq \frac{1}{2}(2) = 1$.

In all cases where at least one capture is possible, we have $\frac{1}{2}(W) + S \geq 1$, exactly what we needed. Thus: $h_mobility(s) = I + \frac{1}{2}(W) \leq (I + W + S) - 1 = k - 1$.

Combine with the Cost Lower Bound

Putting it all together, from the game dynamics we have that $C^*(s) \geq k(s) - 1$, and from the graph argument we have that $h_mobility(s) \leq k(s) - 1$.

Therefore, for every non-goal state where a move is possible, $h_mobility(s) \leq C^*(s)$. For unsolvable states, $C^*(s) = \infty$, so the inequality trivially holds. For the goal state (one piece left), you would typically override and define $h(goal) = 0$ by convention. So yes, h_mobility is an admissible heuristic for this puzzle.

Checking Consistency

Consider the position in Figure 2 and because the queen is the only piece that can attack, the next move is going to be it taking the pawn on E5.

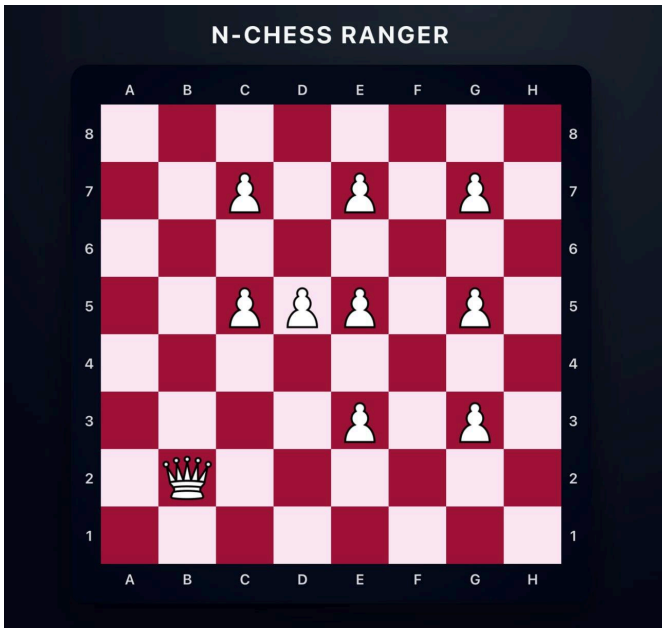

Figure 2: An example of an initial state

Assume the state in Figure 2 is s. The h_mobility(s) = 8 + ½ (2) = 9 (we have 8 isolated pawns, a queen that is attacking a pawn and not being attacked, and a pawn not attacking any piece but being attacked by a queen).

Let's denote the state after the capture (the queen on E5 and the pawn removed) as s'. Then h_mobility(s') = 1 + ½ (7) = 4.5 (we have an isolated pawn on C5, the remaining pawns are attacked by the queen and the queen only without attacking other pieces, the queen is strongly connected).

Because we have a unit step cost of 1, meaning C(s, s', a) = 1, we can say that for consistency h_mobility(s) - h_mobility(s') $\leq$ C(s, s', a) = 1 $\Rightarrow$ 9 - 4.5 $\leq$ 1 $\Rightarrow$ the inequality does not hold, meaning that the heuristic is not consistent.

This means that the h_mobility is admissible (never overestimates cost to goal), but it is not consistent, because some captures can drastically increase connectivity and make h drop by more than 1 in a single step.

The same follows for h_isolated, it is admissible, but inconsistent. Unfortunately, we were unable to come up with a consistent heuristic, so we decided to stick with h_mobility. We chose the mobility heuristic over the isolated because if there is at least one weakly connected piece on the board, then the mobility heuristic is always greater, if not, then it is as good as the isolated heuristic, meaning that the mobility heuristic dominates the isolated heuristic, therefore better to use.

3.3 Designing a GUI

We designed a user interface using Python for running experiments and tracking how algorithms perform given configurations of different numbers of pieces. The user can generate a random state, containing the number of pieces of their choice (4-10). The user can choose which algorithm they want to solve the puzzle or at least try to solve. When an algorithm is executed, the interface shows the capturing moves and records metrics and the moves executed. The metrics that are recorded are the number of expanded nodes, number of generated nodes, max depth and runtime. There are three modes to the interface, which the user can choose, piece placement mode, piece deletion mode, and manual solve mode. This means that the interface allows the user to create custom configurations and run algorithms. Additionally, the user can try to solve configurations manually. This design helped us debug and fix our algorithm implementation and also conduct experiments in order to compare the algorithms based on the metrics we discussed.

4 Experiments, Results and Analysis

4.1 Experiments

4.1.1 Small Configuration Experiment

Since we designed an interface, where configurations of 4-10 pieces can be generated, we decided to separate it into two sections. We will call 4-7 piece configurations small, and the other one large. For this experiment, we decided to generate 30 different solvable puzzles of size 6 and recorded algorithmic performances. The metrics recorded are number of expanded nodes e, number of generated nodes g, maximum depth d, and runtime in milliseconds t.

From our sample we calculated the average metrics shown in Table 1.

	e	**g**	**d**	**t**
DFS	16.2	16.2	5	~ 8.9
A*_I	160.6	346.1	5	~72.47
A*_M	160.6	346.1	5	~57.29
HC	~2	~15.4	~1	~3.91
SHC	3.8	~8.9	2.23	~3.4
SA	6.1	20.9	5.4	~5.42

Table 1. Average Values for Small Configuration Sample

4.1.2 Large Configuration Experiment

As we discussed in the previous section the configurations with 8-10 pieces are considered large and we conducted an experiment with 30 randomly generated solvable sample puzzles with 10 pieces on the board. Table two shows the averages of the same metrics discussed before.

	e	g	d	t
DFS	261.5	261.5	9	19.19
A*_I	16706.86	64909.6	9	~1334.7
A*_M	16706.86	64909.6	9	~1360.8
HC	2.6	41.7	1.63	~9
SHC	6.63	21.5	~5.5	~11.3
SA	10.13	56	10.6	~13.8

Table 2. Average Values for Large Configuration Sample

Additionally, for large configurations we decided to observe SHC and SA when they do find a solution, because most of the time they fail to find a solution and it is interesting how they perform when they manage to find a solution. Table 3 represents metrics when SHC and SA find a solution.

	e	g	d	t
SHC	10	22.3	9	11.95
SA	21	76.6	20	17.4

Table 3. Average Values when SHC and SA Find a Solution on a Large Configuration

4.2 Results

After running the experiments, we can infer the following results for the algorithm performance.

4.2.1 Success Rate

We plotted the success rate of each algorithm to understand which one is best at finding a solution when one exists. The result can be seen in Chart 1. Since we observed really similar results, we decided to only include the large configuration to avoid repetition.

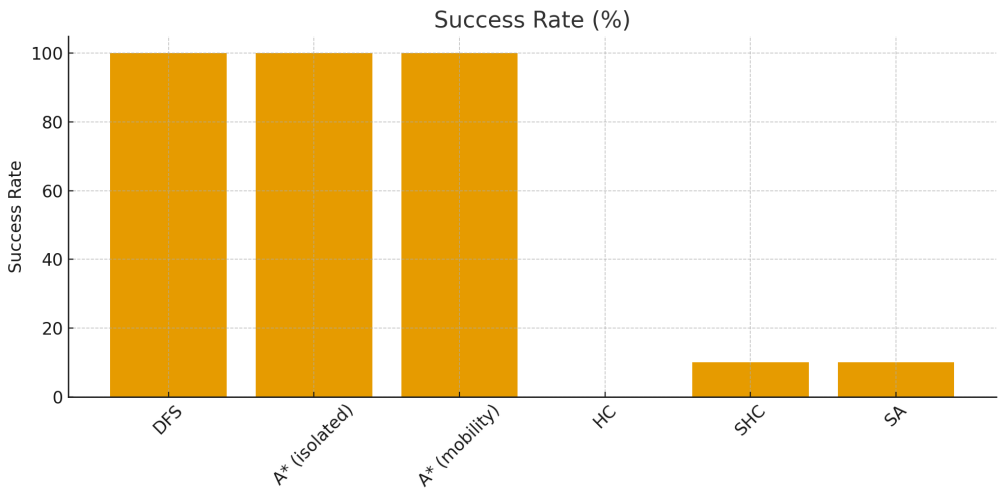

Chart 1. Success Rate of Algorithms

As we can see in Chart 1, the success rate for DFS and A* with both heuristics is 100%, meaning that if the puzzle has a solution, then these algorithms will find them. This shows that for this puzzle, these algorithms are reliable. In contrast, Hill-Climbing never found a solution, meaning that it got stuck on local minima and could not find a better neighbor to move on to. Stochastic Hill-Climbing and Simulated Annealing solved 3 out of 30 puzzles, both showing 10% success rate. This shows that compared to Hill-Climbing, algorithms perform better, but surely perform worse compared to DFS and A*.

4.2.2 Runtime

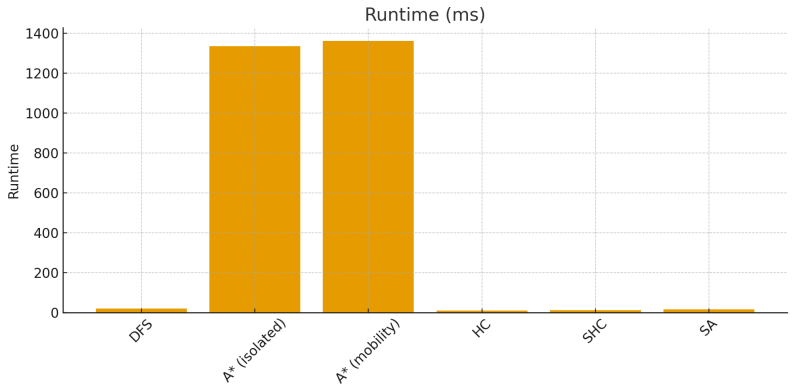

Chart 2. Runtime of Algorithms

Considering the fact that DFS finds a solution 100% of the time, it has really low runtime, which is only ~19 milliseconds, which makes it both fast and reliable. In comparison, A* solves the puzzle in ~1347 milliseconds, which is almost 70 times slower than DFS. The huge difference can be explained by the fact that both heuristics for A* are inconsistent, hence it can force A* to re-expand the nodes, which requires more time.

The slow runtimes for other algorithms are due to the reason that they terminate early, since they do not find a solution. Hence, the low runtime can be misleading.

However, to be fair, we also computed the runtimes when Stochastic Hill-Climbing and Simulated Annealing found a solution, and for Stochastic Hill-Climbing the mean runtime was 11.95, and for Simulated Annealing it was 17.4. Looking at the results, we can conclude that when Stochastic Hill-Climbing and Simulated Annealing find a solution, they perform even better than DFS. But considering that their success rate is only 10%, they cannot be considered reliable.

4.2.3 Number of Expanded Nodes

Since the number of generated nodes was too large we decided to log-scale it to make it readable. Chart 3 shows the results.

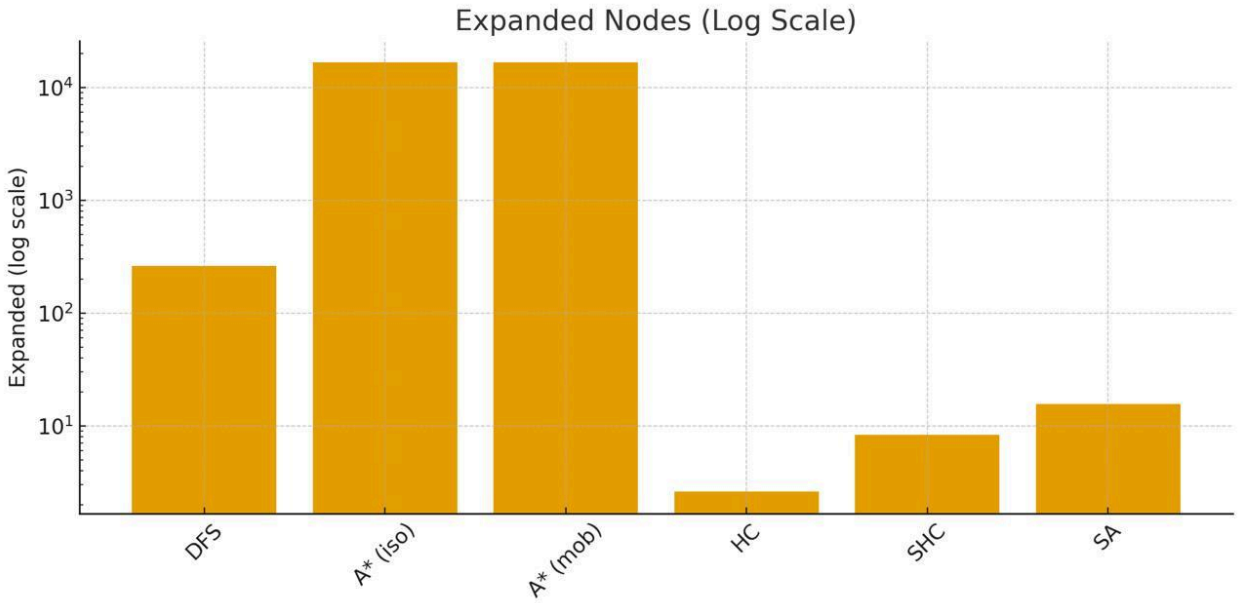

Chart 3. Log-Scaled of the Number of Expanded Nodes

Using a log-scale graph allows us to see the true magnitude differences in expanded nodes. A* (both isolated and mobility) expands around 16,700 nodes, which completely dominates the graph. DFS, in contrast, expands only ~262 nodes, roughly 60 times fewer.

Local search methods expand almost no nodes in comparison. HC expands barely 2–3 nodes, and the other two expand between 8 and 16 nodes. Again, this is not a sign of efficiency. They expand so few nodes because they immediately get stuck and quit without searching much.

This graph visually demonstrates the inefficiency of A*, despite being a heuristic-guided algorithm, inconsistent heuristics cause it to explore huge portions of the state space. DFS, on the other hand, expands far fewer states and still reaches the goal every time.

4.2.4 Number of Generated Nodes

Here we looked at the nodes generated by each algorithm, again log-scaling them. Chart 4 shows the results.

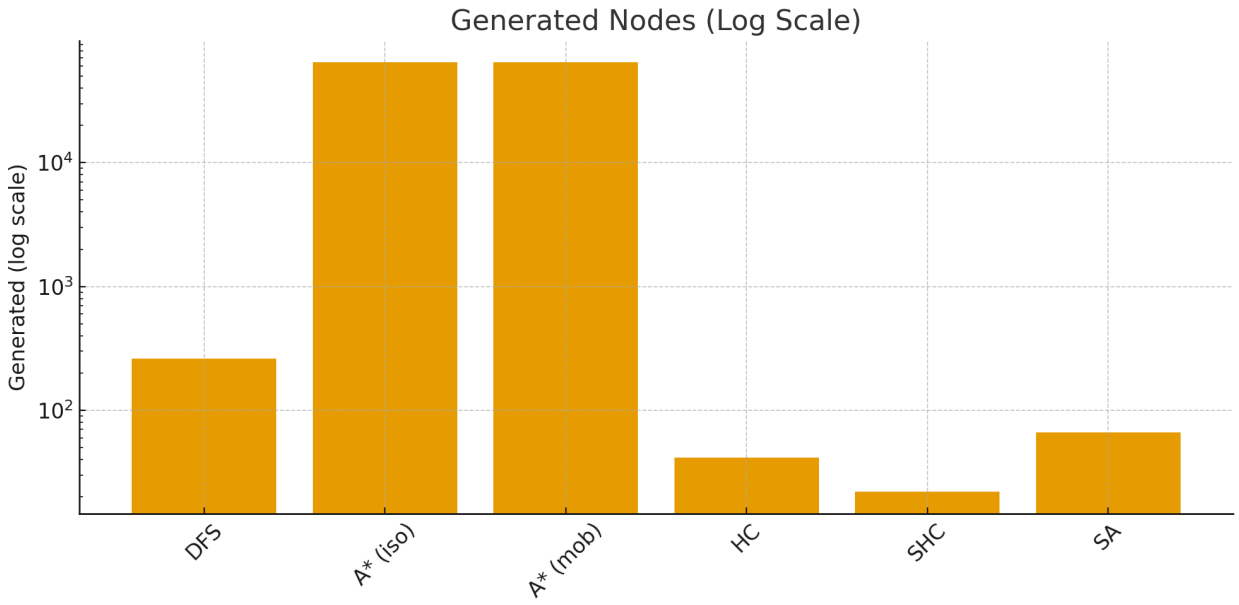

Chart 4. Log-Scaled of the Number of Expanded Nodes

The generated nodes chart follows the same pattern as expanded nodes but with even more dramatic scale differences. A* generates about 65,000 states, by far the largest among all algorithms. DFS generates only about 261 states, meaning DFS generates 250 times fewer states than A* while still finding the solution.

Local search algorithms generate fewer than 100 nodes, again because they only explore shallow neighborhoods before failing.

This chart illustrates the exact reason why DFS outperforms A*. A*'s heavy reliance on heuristic evaluation combined with heuristic inconsistency causes excessive branching and re-generation of states. DFS remains efficient because it simply follows depth-first exploration with minimal overhead.

5 Conclusion

To conclude, looking at our results, a clear picture emerges about how these algorithms behave on the Chess Ranger puzzle. DFS turned out to be the most dependable and practical method. It found a solution every single time and did surprisingly quickly, even as the puzzle became larger. A* also guaranteed to find a solution, struggled a lot in practice because the heuristics were inconsistent.

The local search method behaved oppositely. They were extremely fast and generated very few nodes, but only because they rarely got beyond the first few steps of the puzzle. Hill-Climbing never found a single solution, and while Stochastic Hill-Climbing and Simulated Annealing succeeded occasionally, their success rate was far too low to rely on. Even though they were fast when they worked, they almost never worked.

Overall, the experiments show that algorithms which thoroughly explore the state space are good for Chess Ranger. DFS consistently gave the best combination of speed and reliability, while A* only performed well in theory, not in practice. Local search methods, despite their efficiency, simply could not handle the structure of this puzzle. In short, if you want an algorithm that actually solves Chess Ranger, DFS is the clear winner.

5.1 Difficulties Encountered

Throughout the project we encountered several challenges that slowed our progress. The most persistent difficulty was designing a meaningful heuristic. Although we improved the isolated heuristic and developed the mobility heuristic, we were unable to create a consistent one despite multiple attempts.

Implementing the search framework also proved challenging. Early versions produced many errors, and ensuring that the algorithms behaved correctly and did not over-expand the search space required extensive debugging. Developing the interactive interface posed a similar issue, as we were unfamiliar with interface design and had to learn the necessary tools from scratch. Small visual or functional problems often took a lot of time to fix.

Another obstacle was the lack of existing solvers or literature on the Chess Ranger puzzle. With no established heuristics or reference solutions, validating our results depended entirely on our own testing and reasoning.

Finally, time constraints during midterms and overlapping coursework added additional pressure and required careful coordination within the team.

Despite these challenges, this project was a valuable experience in algorithm design, heuristic development, debugging, and solving a problem for which no standard solution existed. These difficulties helped us understand search algorithms and their behavior more deeply, especially in constrained and non-traditional puzzle environments.

5.2 Future Research Directions

A few promising extensions of this work can be examined. One direction is to come up with a consistent heuristic. Not only is our heuristics inconsistent, our results show that DFS outperforms A* search. A good breakthrough would be finding a heuristic that makes A* work as good as or better than DFS.

A future work may also include different search strategies, including Monte Carlo simulations or other methods to find a method that can greatly improve computational efficiency.

6 References

[1] Aravind, N. R., Misra, N., & Mittal, H. (year). *Chess is hard even for a single player*. Department of Computer Science and Engineering, Indian Institute of Technology. Retrieved November, 2025, from https://arxiv.org/abs/2203.14864

[2] Puzzle-Chess. (n.d.). *Chess Ranger – online puzzle game*. Retrieved November 13, 2025, from https://www.puzzle-chess.com/chess-ranger-4/

[3] Maßberg, J. (2015). Solitaire Chess is NP-complete. ArXiv, abs/1501.06398. Retrieved November 11, 2025, from https://arxiv.org/abs/1501.06398v1

[4] Thaihoaho. (March 2025). *ChessRangerSolver* [Source code]. GitHub. Retrieved November 10, 2025, from https://github.com/thaihoaho/ChessRangerSolver